Website identity — implementation spec

For: website backend and frontend developers
Status: ready to build
Scope: everything needed to identify a visitor across visits, devices and channels, and to give our support bot useful context about them.

This document is written to be implementable as-is. Schema, function contracts, API endpoints, frontend call sites and test cases are all specified. Where a decision is yours to make, it says so explicitly.

Read §1 before §2. The design has a few choices that look over-engineered until you know what they're protecting against — and two failure modes that produce no error at all when you get them wrong.

---

1. Why this exists

The problem, stated plainly

Today, every visit to emotorad.com is a stranger. We have no way to know that the person reading the Doodle page this morning is the same person who compared three e-cycles last week, or the same person who messages us on WhatsApp tomorrow.

That gap costs us in a way that's specific to how Emotorad actually sells. Roughly 85% of our volume goes through 350–600 dealers, not through website checkout. The website is largely a *discovery* surface that hands the customer off — to WhatsApp, to a phone call, to a showroom. So the most valuable information we ever collect (what someone researched, what they compared, what they were about to spend) is generated online and then **thrown away at exactly the moment it becomes useful**.

What that looks like concretely

A real journey, and what we currently retain from it:

	WHAT HAPPENS	WHAT WE KNOW
10 Aug	Lands from an Instagram ad on a laptop. Compares three e-cycles. Leaves.	Nothing
15 Aug	Returns on their phone. Looks at two more. Opens the EMI page.	Nothing
20 Aug	Messages WhatsApp: "which is the nearest dealer?"	A phone number

The dealer receives a phone number and no idea this person spent two sessions comparing models in the ₹30–40k band and checked the EMI terms. Our support bot opens with "how can I help you?" to someone we could have greeted with "still deciding between the Doodle and the Trex?"

After this work, the same journey ends with the WhatsApp conversation carrying that entire history — because the customer reached WhatsApp by tapping a button on our own site, and that button carried a reference code.

The three things it unlocks

Our AI support bot gets context. It's being built in parallel and reads `GET /api/identity/context` (§8) at the start of every conversation. Without this work it opens every chat blind. With it, it knows what the

person owns, what they've been looking at, and what we last spoke about — across WhatsApp, the website and phone calls.

Dealers get a lead brief instead of a phone number. "Viewed EMX Plus four times and Doodle V3 twice over two visits, ₹30–40k band, opened the EMI page, wants a dealer in Baner" is a materially different handover from a bare number.

The homepage can personalise to what someone actually looked at last visit.

Why the timing is non-negotiable

This has to land **during the revamp, not after**. A cookie you didn't set on launch day cannot be backfilled — every day without it is a day of visitor behaviour that is permanently unattributable. There is no migration that recovers it later.

Where your work fits

You're building one half of a system. The other half is an AI support agent that reads from these tables. That's why §8's context endpoint matters more than it might look, and why `verified_phone` in its response is load-bearing: it's what tells the bot whether it may say a customer's name out loud, or must stay generic because a cookie identifies a *browser* and the person at the keyboard might be someone else in the household.

Two places where precision really matters

Most bugs announce themselves. These two don't:

Phone normalisation (§5.2). If `9876543210` and `+919876543210` both get stored, one person silently becomes two, their history splits in half, and nothing anywhere raises an error. You would find out months later, from duplicate customers in reporting.

The unverified-merge rule (§5, branch 3). If we merged on an unverified email, one person mistyping someone else's address fuses two customers' identities — their browsing, their conversations, potentially their purchase history visible to each other. That's a privacy incident, not a bug, and it is unpickable after the fact.

Both are three lines of code. Both are the reason this document is longer than it looks like it needs to be.

1.1 What you're building, in one paragraph

A first-party cookie holding a UUID, minted **server-side** on a visitor's first request. A table that maps that cookie — plus phone numbers, WhatsApp IDs and emails — to a `cluster_id` representing one person. A small events table recording the handful of clicks that indicate buying intent. And a few endpoints that link identifiers together when a visitor identifies themselves.

Out of scope: device fingerprinting, probabilistic matching, call tracking, buying a CDP.

---

2. Why server-side, and why not Google Analytics

We already run GA4, and it sets a `_ga` cookie. It is not usable as the spine here:

- **GA's cookie is set by JavaScript.** Safari's ITP caps JS-set cookies at roughly 7 days, so iPhone visitors would look new every week. A cookie set by our own server via a `Set-Cookie` header is not subject to that cap. *(Re-verify current ITP behaviour when you build — Apple changes it.)*
- **Ad blockers block GA heavily.** Those visitors would have no ID at all.
- **GA has no user-level real-time API.** User-level rows require the BigQuery export, and the free tier is a daily batch — useless for a live conversation.
- **Google's terms prohibit sending PII to GA**, so the phone-to-cookie join must happen our side anyway.

GA keeps doing marketing reporting. We capture its client ID as one more row in our table so GA data can be joined to the right person later, but nothing depends on it.

---

3. The cookie

PROPERTY	VALUE
Name	`em_aid`
Value	UUIDv4
Set by	Server, `Set-Cookie` header, on the first request where it is absent
`Domain`	`.emotorad.com`
`Max-Age`	`63072000` (2 years)
`SameSite`	`Lax`
`Secure`	yes
`HttpOnly`	**yes** — see below

Decision — `HttpOnly`. Set it. It stops XSS stealing the ID. The frontend still needs the value for event calls, so the server renders it into the page:

```
<meta name="em-aid" content="{{ em_aid }}">
```

Frontend reads `document.querySelector('meta[name="em-aid"]').content`. Do **not** drop `HttpOnly` for convenience.

Middleware

On every HTML request, before render:

```
em_aid = request.cookies.get("em_aid")
if not em_aid:
    em_aid = uuid4()
    response.set_cookie("em_aid", em_aid, domain=".emotorad.com",
                        max_age=63072000, samesite="Lax",
                        secure=True, httponly=True)
request.state.em_aid = em_aid        # available to handlers and templates
```

Skip known bot user-agents so the table doesn't fill with crawler rows.

Note: creating the cookie does *not* create a database row. Rows are created on first `link_identity` call (§5) or first event (§6). A visitor who loads one page and leaves costs us nothing.

---

4. Schema

```
-- One row per identifier. Many rows share a cluster_id = one person.
create table identities (
  id             bigserial primary key,
  cluster_id     uuid not null,
  identity_type  text not null,          -- anon_id | phone | email | wa_id
                                         -- | ga_client_id | user_id
                                         -- NOT frame_number (see note below)
  identity_value text not null,
  verified       boolean not null default false,
  first_seen_at  timestamptz not null default now(),
  last_seen_at   timestamptz not null default now(),
  unique (identity_type, identity_value)
);
create index on identities (cluster_id);

-- Audit trail of merges. Never delete from this.
create table cluster_merges (
  id           bigserial primary key,
  from_cluster uuid not null,
  to_cluster   uuid not null,
  reason       text not null,
  merged_at    timestamptz not null default now()
);

-- Buying-intent events only. Not a general analytics firehose.
create table events (
  id         bigserial primary key,
  em_aid     uuid not null,              -- keyed on the COOKIE, not cluster_id
  event_name text not null,
  properties jsonb not null default '{}',
  occurred_at timestamptz not null default now()
) partition by range (occurred_at);
create index on events (em_aid, occurred_at desc);

-- Monthly partitions. Dropping one is instant; never DELETE from this table.
create table events_2026_08 partition of events
  for values from ('2026-08-01') to ('2026-09-01');

-- SAFETY NET: without this, an insert with no matching partition FAILS.
-- Rows landing here mean the partition job didn't run — alert on it being non-empty.
create table events_default partition of events default;

-- Short code -> cookie, for the WhatsApp click-through (§9).
create table whatsapp_refs (
  code       text primary key,          -- 6 chars, e.g. '7KQ2M9'
  em_aid     uuid not null,
  created_at timestamptz not null default now(),
  expires_at timestamptz not null default now() + interval '24 hours'
);
create index on whatsapp_refs (expires_at);

-- Lazily-populated cache of assembled context. NOT precomputed for everyone.
create table profile_cache (
  cluster_id  uuid primary key,
  payload     jsonb not null,
  computed_at timestamptz not null default now()
);
```

Partition maintenance — do not skip this

`events` is partitioned by month, and **a missing partition makes inserts fail outright.** Without maintenance, event capture breaks at midnight on the 1st of a month. Two things are required:

1. A scheduled job that creates next month's partition ahead of time and drops partitions older than 90 days. Either `pg_partman`, or a monthly cron running `create table ... partition of` and `drop table`.
2. The `events_default` partition above as a backstop, with an alert if it ever holds rows — that means the job failed and you're an outage away from losing events.

Two schema decisions worth understanding

Events are keyed on `em_aid`, not `cluster_id`. Cluster IDs get retired when two clusters merge; cookie values never change. Keying on the cookie and resolving to a cluster at *read* time means merges apply retroactively to all historical events with zero data migration. Keying on `cluster_id` would mean rewriting history on every merge.

Frame numbers are deliberately absent. Bike ownership lives in the OMS warranty table, keyed on phone, and stays there as the single source of truth. This table maps `cluster_id ↔ phone`; ownership is one further hop to the OMS. Copying frame numbers here would mean a dealer registers a warranty and our table is stale until the next sync.

---

5. `link_identity` — the one function that matters

Called whenever a new identifier becomes known. **Not on every page load.** A visitor who reads thirty pages triggers it once, on their first request.

```
def link_identity(em_aid, identity_type, identity_value, verified) -> uuid:
    """Returns the cluster_id this identifier now belongs to.
    em_aid may be None — e.g. a WhatsApp message with no ref code.
    MUST run inside a single transaction."""

    identity_value = normalise(identity_type, identity_value)   # §5.2 — always first

    with transaction():
        existing = select_one(
            "select * from identities where identity_type=%s and identity_value=%s",
            identity_type, identity_value)

        # No cookie at all (plain WhatsApp / IVR contact). There is no browser
        # to link to, so the identifier stands alone.
        if em_aid is None:
            if existing:
                return existing.cluster_id
            new_cluster = uuid4()
            insert_identity(new_cluster, identity_type, identity_value, verified)
            return new_cluster

        my_cluster = cluster_for(em_aid)     # see below

        # 1. Never seen this identifier — attach it to this browser's person.
        if existing is None:
            insert_identity(my_cluster, identity_type, identity_value, verified)
            return my_cluster

        # 2. Known, same person — common case, keep it cheap.
        if existing.cluster_id == my_cluster:
            update("update identities set last_seen_at=now(), verified = verified or %s "
                   "where id=%s", verified, existing.id)
            return my_cluster

        # 3. Known, DIFFERENT person — two clusters are actually one human.
        if verified:
            survivor, loser = older_of(existing.cluster_id, my_cluster)
            update("update identities set cluster_id=%s where cluster_id=%s", survivor, loser)
            insert("insert into cluster_merges (from_cluster, to_cluster, reason) "
                   "values (%s,%s,%s)", loser, survivor, f"{identity_type} verified")
            delete("delete from profile_cache where cluster_id in (%s,%s)", survivor, loser)
            return survivor

        # Unverified identifier claiming an existing person: DO NOT merge.
        # This is the typo'd-email protection.
        log.warning("unverified collision", extra={"em_aid": em_aid, "type": identity_type})
        return my_cluster
```

Survivor rule: the cluster with the earlier `first_seen_at` wins, so merges are deterministic rather than dependent on which request arrived first.

Invalidate `profile_cache` on merge — otherwise the bot serves pre-merge context.

5.1 The four helpers, specified

`link_identity` calls these. Don't leave them to interpretation — `cluster_for` in particular does the heaviest lifting and has a race condition in the obvious implementation.

```
def cluster_for(em_aid) -> uuid:
    """Cluster this browser belongs to, creating one on first sight.
    The ON CONFLICT is not optional: two concurrent first requests from the
    same new browser would otherwise create two clusters, or crash on the
    unique constraint."""
    row = select_one("select cluster_id from identities "
                     "where identity_type='anon_id' and identity_value=%s", em_aid)
    if row:
        return row.cluster_id

    new_cluster = uuid4()
    insert("""insert into identities (cluster_id, identity_type, identity_value, verified)
              values (%s, 'anon_id', %s, false)
              on conflict (identity_type, identity_value) do nothing""",
           new_cluster, em_aid)
    # Re-select: if a concurrent request won the race, take its cluster.
    return select_one("select cluster_id from identities "
                      "where identity_type='anon_id' and identity_value=%s", em_aid).cluster_id


def older_of(cluster_a, cluster_b) -> tuple[uuid, uuid]:
    """(survivor, loser) — earliest first_seen_at across the cluster's rows wins."""
    rows = select_all("""select cluster_id, min(first_seen_at) as born from identities
                         where cluster_id in (%s,%s) group by 1 order by born asc""",
                      cluster_a, cluster_b)
    return rows[0].cluster_id, rows[1].cluster_id


def signals_from_events(em_aids) -> list[str]:
    """Which intent markers this person has shown, ever, in the retention window."""
    rows = select_all("""select distinct event_name from events
                         where em_aid = any(%s)
                           and event_name in ('emi_page_viewed','test_ride_page_viewed',
                                              'dealer_locator_used','add_to_cart',
                                              'checkout_started')
                           and occurred_at > now() - interval '90 days'""", em_aids)
    return [r.event_name for r in rows]


def phone_if_verified(cluster_id) -> str | None:
    """The verified phone for this person, or None. Drives what the bot may disclose."""
    row = select_one("""select identity_value from identities
                        where cluster_id=%s and identity_type='phone' and verified=true
                        order by first_seen_at asc limit 1""", cluster_id)
    return row.identity_value if row else None
```

5.2 Normalisation — inside the function, never at call sites

TYPE	RULE
`phone`	E.164 with country code: `+919876543210`. Never `9876543210`, never spaced or dashed
`email`	Trim, lowercase
`wa_id`	WhatsApp sends digits with no `+` — convert to the same E.164 form as `phone` so they match
`anon_id`, `ga_client_id`, `user_id`	Verbatim

Skipping this is the design's silent failure: the unique constraint stops deduplicating, one person becomes several clusters, and **nothing raises an error**.

---

6. Events to capture

Eight events, not everything. These are the ones that indicate buying intent.

EVENT_NAME	PROPERTIES
`product_viewed`	`{model, price, category}`
`category_viewed`	`{category}`
`price_filter_applied`	`{min, max}`
`emi_page_viewed`	`{}`
`test_ride_page_viewed`	`{}`
`dealer_locator_used`	`{pincode}`
`add_to_cart`	`{model, price}`
`checkout_started`	`{cart_value}`

Frontend posts to `POST /api/events` with `{event_name, properties}`; the server reads `em_aid` from the cookie. **Do not** accept `em_aid` from the request body — it must come from the cookie, or anyone can write events against another person's ID.

---

7. Endpoints to build

ENDPOINT	CALLED BY	DOES
`POST /api/events`	Frontend (public)	Insert an event. `em_aid` from cookie only. **Rate-limit it** — it's unauthenticated and writes rows
`POST /api/identity/ga`	Frontend (public)	`link_identity(em_aid, 'ga_client_id', <_ga value>, false)`. Rate-limit
`GET /api/whatsapp/ref`	Frontend (public)	Returns a short code mapped to this `em_aid` (§9). Rate-limit
`POST /api/identity/resolve`	**Internal — the bot backend**	**`{identity_type, identity_value}` → `{cluster_id}`.** How the bot gets a `cluster_id` in the first place
`POST /api/identity/link`	**Internal**	Called by our handlers after OTP/login, and by the bot for the WhatsApp `ref:` stitch
`GET /api/identity/context?cluster_id=`	**Internal** — the bot backend	Returns assembled context (§8)
`DELETE /api/identity/erase`	**Internal** — support/legal tooling	Erases a whole cluster (§10)

`/api/identity/resolve` is what joins the two halves of this system. The bot never has a `cluster_id` — it has a phone number (WhatsApp, IVR) or a cookie (website chat). It calls `resolve` first, then `context` with what comes back. Without it there is no path from an inbound message to a person, and the whole thing is inert.

```
WhatsApp message  → resolve {wa_id, "+9198..."}   → cluster_id → context
Website chat      → resolve {anon_id, "<cookie>"} → cluster_id → context
```

`resolve` creates a cluster if the identifier is new, so it never returns empty — a first-time contact is a valid person with no history.

The three internal endpoints must not be reachable from the public internet: they read and delete personal data across customers with no per-user authorisation.

Existing handlers to modify — add one `link_identity` call to each success path:

```
# OTP verification (registration, test-ride booking, checkout)
if otp_service.check(phone, otp):
    link_identity(request.state.em_aid, "phone", phone, verified=True)

# Login
link_identity(request.state.em_aid, "user_id", user.id, verified=True)

# Any form capturing an email
link_identity(request.state.em_aid, "email", email, verified=False)   # never merges
```

8. The context endpoint

The bot calls this once at conversation start. **Compute lazily and cache** — do not build a scheduled job that precomputes profiles for every visitor. Most visitors never chat with us, so precomputing for everyone is ~97% wasted work, and a nightly rollup would miss the browsing someone did five minutes before opening the chat, which is the most relevant browsing there is.

```
def get_context(cluster_id) -> dict:
    cached = select_one("select payload from profile_cache "
                        "where cluster_id=%s and computed_at > now() - interval '30 minutes'",
                        cluster_id)
    if cached:
        return cached.payload

    em_aids = select_all("select identity_value from identities "
                         "where cluster_id=%s and identity_type='anon_id'", cluster_id)

    top_products = select_all("""
        select properties->>'model' as model, count(*) as views, max(occurred_at) as last_seen
        from events
        where em_aid = any(%s)
          and event_name = 'product_viewed'
          and occurred_at > now() - interval '90 days'
        group by 1 order by views desc limit 5
    """, em_aids)

    payload = {
        "top_products": top_products,
        "signals": signals_from_events(em_aids),   # emi_viewed, test_ride_viewed, etc.
        "visit_count": ...,
        "verified_phone": phone_if_verified(cluster_id),   # null if none
    }
    upsert("insert into profile_cache (cluster_id, payload, computed_at) "
           "values (%s,%s,now()) on conflict (cluster_id) do update set "
           "payload=excluded.payload, computed_at=now()", cluster_id, payload)
    return payload
```

That aggregation is a single indexed query over a few hundred rows — tens of milliseconds, not a data-warehouse job.

`verified_phone` is load-bearing. The bot uses it to decide what it may say out loud: with a verified phone it can state the customer's name, bikes and warranty; without one it may only reference product interest. A cookie identifies a browser, not a person — shared family laptops are common, and "Hi Ananya, about your EMX Plus?" to her husband is a data leak.

---

9. The WhatsApp reference code — highest-value item here

When a visitor taps "chat on WhatsApp", append a short code to the prefilled message:

```
https://wa.me/<number>?text=Hi%2C%20I%27d%20like%20to%20know%20more.%20%5Bref%3A7KQ2M9%5D
```

`GET /api/whatsapp/ref` generates a short code (6 chars, not the raw UUID — a UUID in a customer-visible message looks broken), stores `code → em_aid` with a 24-hour TTL, and returns it. The frontend builds the

link with it.

The bot backend parses `ref:` from the first inbound message and calls `link_identity(em_aid_from_code, 'wa_id', <sender>, verified=True)`.

This single mechanism converts an anonymous browser into a verified phone number **with the browsing history already attached** — the most valuable link available, and a frontend change rather than an architectural one.

Verify current behaviour against Meta's docs before building: plain `wa.me` links carry prefilled text only, while Click-to-WhatsApp ads deliver a structured `referral` object.

---

10. Consent and erasure

India's DPDP Act applies. Two things:

- Get legal's read on whether `em_aid` needs consent — a first-party functional cookie is a different question from analytics tracking, and the answer affects where in the page lifecycle it can be set.
- **Erasure must propagate across the whole cluster.** Build `DELETE /api/identity/erase` that takes a `cluster_id`, deletes all its `identities` rows, all `events` for its `em_aid`s, and its `profile_cache` row. Straightforward now, awkward to retrofit.

---

11. Acceptance tests

Automated — these belong in CI

1. First request to any page sets `em_aid`; a second request reuses the same value.
2. The value is identical across `www.emotorad.com` and any subdomain.
3. `link_identity` with a new phone creates one row sharing the browser's `cluster_id`.
4. `link_identity` with a phone already on **another** cluster, `verified=True`, merges the two and writes a `cluster_merges` row; the **older** cluster survives.
5. Same, with `verified=False`, does **not** merge.
6. `link_identity` called twice with identical arguments produces one row, not two.
7. `9876543210` and `+91 98765 43210` resolve to the same row.
8. `link_identity` with `em_aid=None` (plain WhatsApp contact) creates a standalone cluster and does not raise.
9. Two concurrent `cluster_for` calls for the same new `em_aid` produce **one** cluster, not two.
0. After a merge, `profile_cache` for both clusters is gone.
1. `POST /api/events` with an `em_aid` in the body is ignored — the cookie value is used.
2. `resolve` on an unseen identifier returns a fresh `cluster_id` rather than an error.
3. An insert dated next month lands in a real partition, not `events_default`.

4. Erasing a cluster removes its `identities`, `events` and `profile_cache` rows.

Manual / QA — cannot be automated, but must be checked before launch

5. On a real iOS Safari device, the ID survives more than 7 days.
6. With uBlock Origin enabled, `em_aid` is still set — proves independence from GA.
7. Clicking the WhatsApp button produces a link containing `ref:`, and that code resolves to the right `em_aid` on the bot side.
8. `POST /api/events` with an `em_aid` in the body is ignored — the cookie value is used.
9. Clicking the WhatsApp button produces a link containing `ref:`, and that code resolves to the right `em_aid`.

---

12. Build order

Each step is independently testable:

1. `identities` + `cluster_merges` migrations
2. Cookie middleware — verify in browser dev tools
3. `link_identity` with unit tests for all three branches
4. OTP call site — sign up, confirm both rows share a `cluster_id`
5. `events` table (partitioned) + `POST /api/events` + the eight frontend calls
6. `profile_cache` + `GET /api/identity/context`
7. WhatsApp ref code, coordinated with whoever builds the bot
8. Remaining call sites: login, email forms, GA client ID
9. Erasure endpoint

---

13. One thing to check before the data migration

Separate from this work: we plan to backfill the warranty table so one table can answer "is this person known to us." Before making phone the universal key, run:

```
select phone, count(distinct frame_number) as bikes
from warranty_registrations
group by phone
having count(distinct frame_number) > 3
order by bikes desc;
```

Dealers perform most warranty registrations and frequently enter their own number. If a handful of numbers each own dozens of bikes, those are dealers — merging on them would fuse hundreds of unrelated customers into one profile. This query gates the backfill.